Microprocessor

Chapter 6: Input Output Interfaces

Er. Sachita Nand Mishra

Contents

- Parallel Communication – Introduction and Applications
- Serial Communication
  - ✓ Introduction and Applications
  - ✓ Introduction to Programmable Communication Interface 8251
  - ✓ Basic Concept of Synchronous and Asynchronous Modes
- Simple I/O, Strobe I/O, Single handshake I/O, Double handshake I/O
- 8255A and it's Working
  - ✓ Block Diagram
  - ✓ Modes of Operation
  - ✓ Control Word
- RS-232 – Introduction, Pin Configuration (9 pin and 25 pin) and function of each pin, Interconnection between DTE-DTE and DTE-DCE

Input/ Output Devices

- Input / Output devices are the means through which the microcomputer unit communicates with the outside world.
- The link between the I/O devices and the microprocessor is maintained by a circuitry known as I/O module.
- This circuitry includes the specific interfaces needed for I/O devices as well as control functions that implement the I/O transfers within the computer.
- I/O devices usually are appeared as passive devices which take action only when instructed to do.
- The CPU monitors the status of the I/O devices and selects them according to availability and need.

Input/ Output Devices

- Consider the keyboard as input device and the steps when the key is pressed are
  - Microprocessor detects the key change in status of keyboard i.e. the key is pressed.
  - It receives the encoded information corresponding to pressed key.
  - It checks the validity of required signal.
- Consider the printer as output device
  - Here microprocessor checks ideal condition of printer, if ideal then sends the data to be printed and required command for that.
- For interfacing of typical microprocessor to I/O devices such as keyboard, CRT, printer etc.
- All need I/O interface circuits which are of mainly two types.

Input/Output Interface

- The computer system includes special hardware components between the CPU and peripheral devices to supervise and synchronize all input and output transfer. These components are called interface unit (I/O module).
- There are several problems while we are trying to connect the peripheral devices directly with the CPU. They are:
  - ✓ There is wide variety of peripherals with various methods of operation. It would be impractical to incorporate the necessary logic within the processor to control a range of devices.
  - ✓ The data transfer rate of peripherals is often much slower than that of the memory and processor. Thus, it is impractical to use the high speed system bus to communicate directly with a peripheral.
  - ✓ Peripheral devices often use different data formats and word lengths than the computer to which they are attached.
  - ✓ Peripherals are electromechanical and electromagnetic devices and their manner of operation is different from the operation of CPU and memory, which are electronic devices. Therefore, a conversion of signal values may be required.

Parallel Interface

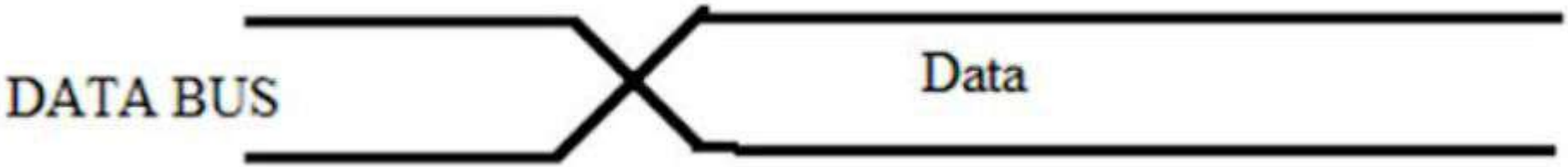
- In **Parallel Transmission**, various bits are sent together simultaneously with a single clock pulse.
- It is a fast way to transmit as it uses many input/output lines for transferring the data.
- N bits of data are handled simultaneously by the bus and the links to the device directly.
- The device which can handle data at higher speed cannot support with serial interface.
- Achieves faster communication but becomes expensive due to need of multiple wires.
- A constant clocking signal is used to provide the timing in parallel communication. The signal is sent with the help of a separate wire within the parallel cable. So we can say parallel communication is synchronous.

Data transfer Modes of Parallel Interfacing

- The information exchanged between a microprocessor and an I/O interface circuit consists of input or output data and control information.
- The status information enable the microprocessor monitor the device and when it is ready then send or receive data. Control information is the command by microprocessor to cause I/O device to take some action.
- If the device operates at different speeds, then microprocessor can be used to select a particular speed of operation of the device.
- The techniques used to transfer data between different speed devices and computer is called synchronizing.
- Different techniques under synchronizing are:
  - ✓ Simple I/O
  - ✓ Simple strobe I/O)
  - ✓ Single Handshaking
  - ✓ Double Handshaking

Simple I/O

- For simple I/O, the buffer switch and latch switches i. e. LED are always connected to the input and output ports.
- The devices are always ready to send or receive data.
- Here cross line indicate the time for new valid data.
- The crossed lines on the waveform represent the time at which a new data byte becomes valid on the output lines of the port.
- The absence of other waveforms indicates that this output operation is not directly dependent on any other signal.



Wait Interface(Simple strobe I/O)

- In this technique, MP need to wait until the device is ready for the operation.
- Strobe signal indicates the time at which data is being activated to transmit.
- In many applications, valid data is present on an external device only at a certain time, so it must be read in at that time.
- E.g. the ASCII-encoded keyboard. When a key is pressed, circuitry on the keyboard sends out the ASCII code for the pressed key on eight parallel data lines, and then sends out a strobe signal on another line to indicate that valid data is present on the eight data lines.
- The sending device, such as a keyboard, outputs a parallel data on the data lines, and then outputs an STB signal to let you know that valid data is present.

Wait Interface(Simple strobe I/O)

- The sending device, such as a keyboard, outputs a parallel data on the data lines, and then outputs an STB signal to let you know that valid data is present.
- For low rates of data transfer, such as from a keyboard to a MP, a simple strobe transfer works well.
- However, for higher speed data transfer, this method does not work because there is no signal which tells the sending device when it is safe to send the next data byte.
- In other words, the sending system might send data bytes faster than the receiving system could read them.
- To prevent this problem, a handshake data transfer scheme is used.

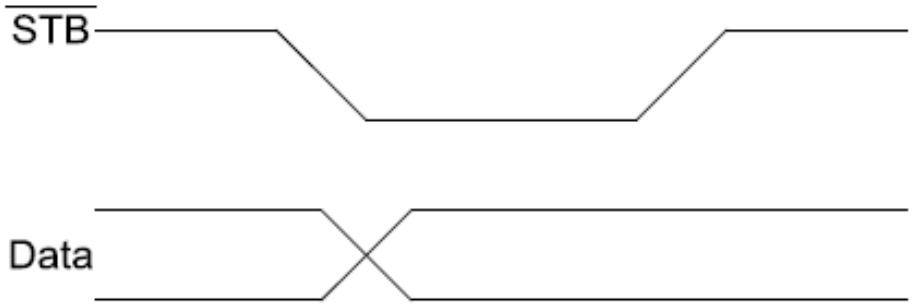


Fig: Simple Strobe I/O

Single Handshaking:

- Strobe is the signal controlled by the sender.
- Acknowledge is a signal controlled by the receiver.
- The peripheral outputs some parallel data and sends an STB signal to the MP.
- The MP detects the asserted STB signal on a polled or interrupts basis and reads in the bytes of data.
- Then, the MP sends ACK (acknowledge) signal to the peripheral to indicate that the data has been read and that the peripheral can send next byte of data.
- The point of this method is that the sending device or system is designed so that it does not send the next byte until the receiving device or system indicates with an ACK signal that it is ready to receive the next byte.
- MP sends or receives data when peripheral is ready.

Single Handshaking:

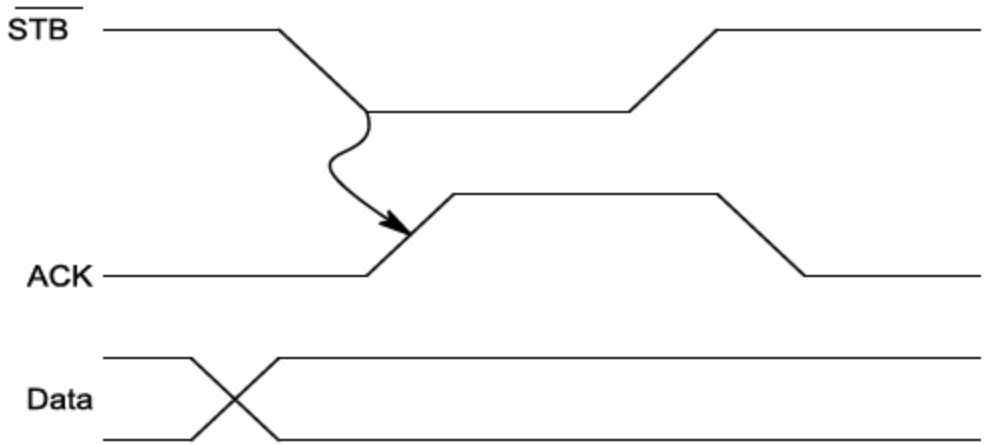


Fig: Single Handshaking

Double Handshaking

- For data transfer where even more coordination is required between the sending system and the receiving system, a double handshake is used.
- The sending (peripheral) device asserts its STB line low to ask the receiving device whether it is ready or not for data reception.
- The receiving system raises its ACK line high to indicate that it is ready.
- The peripheral device then sends the byte of data and raises its STB line high to assure that the valid data is available for the receiving device (MP).
- When MP reads the data, it drops its ACK line low to indicate that it has received the data and requests the sending system to send next byte of data.

Double Handshaking

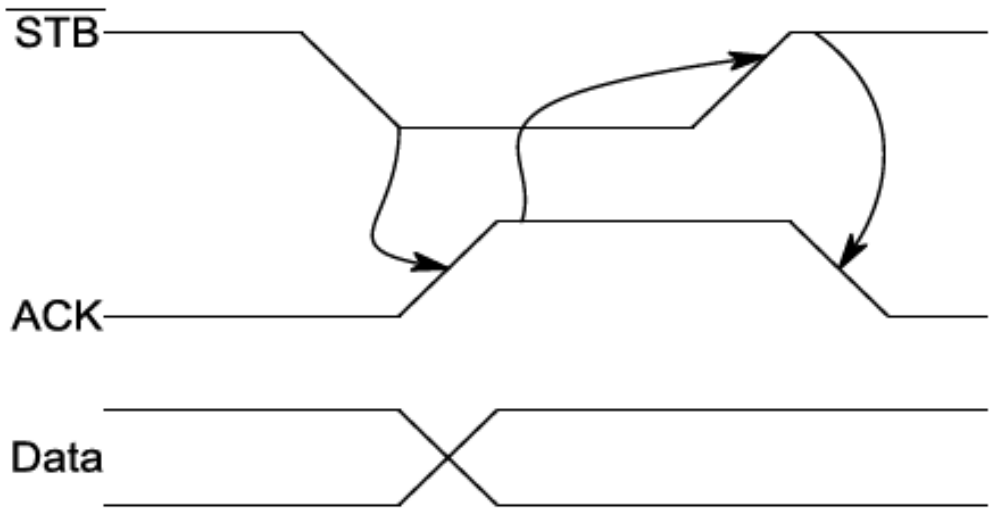


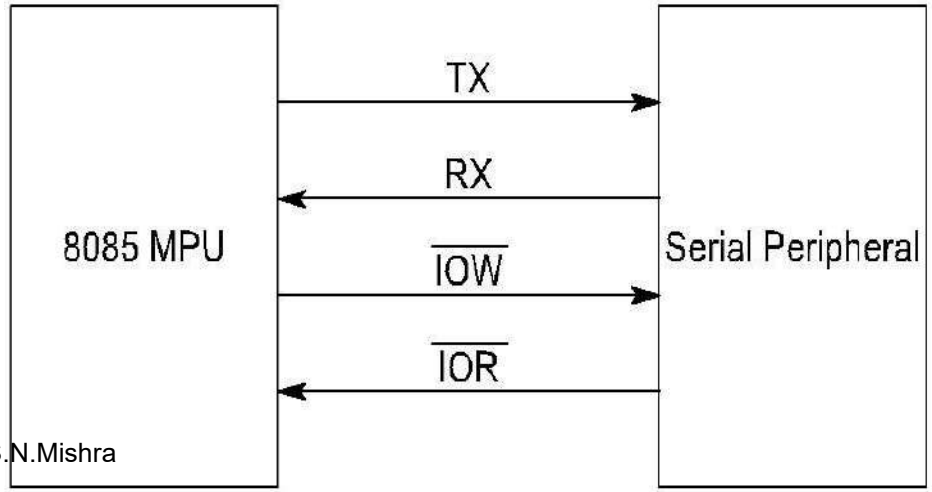
Fig: Double Handshaking

Serial Interface /Serial data transmission:

- Data are transferred serially one bit at a time starting from Least Significant bit.
- Slow due to single communication link but inexpensive to implement.
- It uses clock to separate consecutive bits.
- Its function is to deal with the data on the bus in the parallel mode and communicate with the connected device in serial mode.
- Its data bus has n data lines, the serial I/O interface accepts n bit of data simultaneously from the bus and n bits are sent one at a time thus requiring **n** time slots.
- Not suitable for fast operation needed microprocessor.

Serial I/O transfer is more common than the parallel I/O.
The two major forms of serial data transmission are:

- ✓ **Synchronous serial data transmission:**
- ✓ **Asynchronous serial data Transmission:**



Synchronous Serial data transmission:

- Data is transmitted or received based on a clock signal i. e. synchronously.
- The transmitting device sends a data bit at each clock pulse.
- Usually one or more SYNC characters are used to indicate the start of each synchronous data stream.
- SYNC characters for each frame of data.
- Transmitting device sends data continuously to the receiving device. If the data is not ready to be transmitted, the transmitter will send SYNC character until the data is available.
- The receiving device waits for data, when it finds the SYNC characters then starts interpreting the data.

Synchronous Serial data transmission:

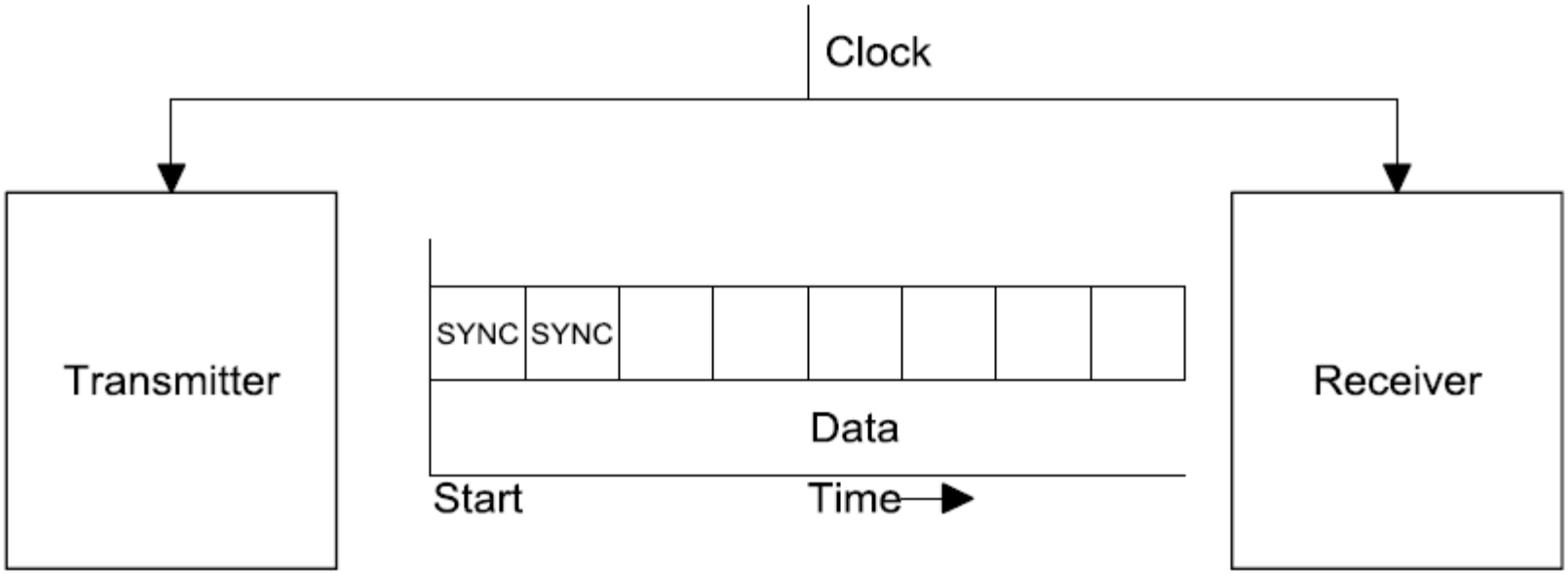


Fig: Synchronous Serial Transmission Format

Asynchronous Serial data Transmission:

- The receiving device does not need to be synchronized with the transmitting device.
- Transmitting device send data units when it is ready to send data.
- Each data unit must contain start and stop bits for indicating beginning and the end of data unit and also one parity bit to identify odd or even parity data.
- For e.g. To send ASCII character (7 bit)
- We need:
  - ✓ 1 start bit: beginning of data
  - ✓ 1 stop bit: End of data
  - ✓ 1 Parity bit: even or odd parity
  - ✓ 7 or 8 bit character: actual data transferred

Asynchronous Serial data Transmission:

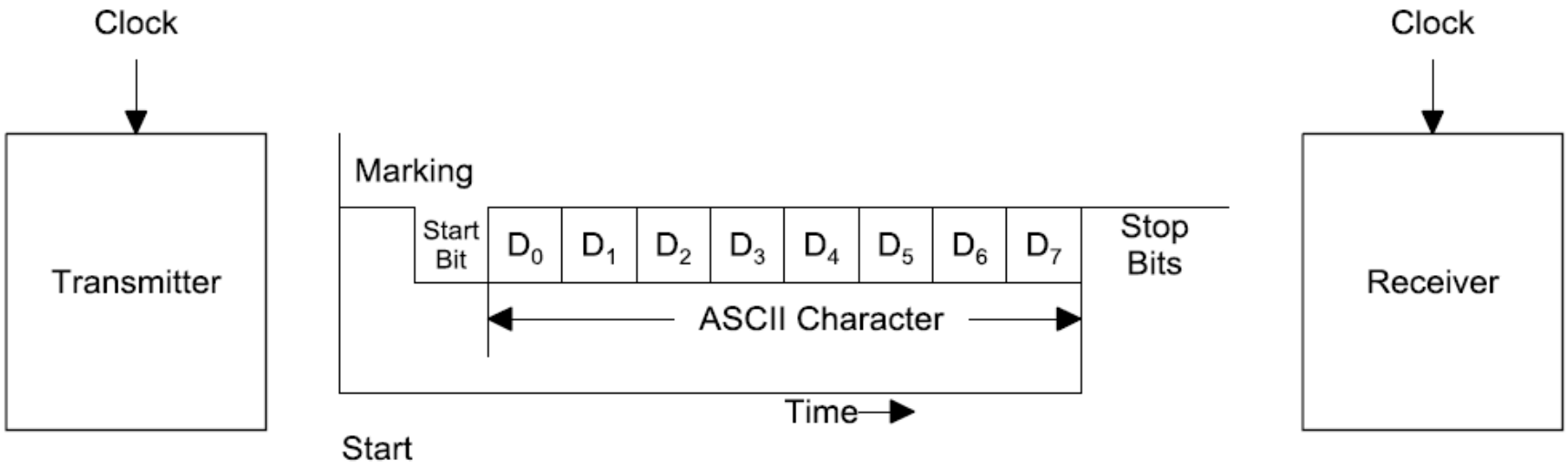


Fig: Asynchronous Serial Transmission Format

Synchronous Vs. Asynchronous Data Transmission

- Asynchronous Communication
  - ✓ Simple interface (limited data rate, typically < 64 kbps)
  - ✓ Used for connecting: Printer, Terminal, Modem, home connections to the Internet
  - ✓ No clock sent (Tx & Rx have own clocks)
  - ✓ Requires start and stop bits which provides byte timing and increases overhead
  - ✓ Parity often used to validate correct reception.
  - ✓ Independent transmit & receive clocks
- Synchronous Communication
  - ✓ Clock sent with data (more configuration options).
  - ✓ Synchronized transmit & receive clocks.
  - ✓ More complex interface (high data rates supported up to ~ 10 Gbps)
  - ✓ Used for: Connections between computer and telephony networks.

Synchronous vs Asynchronous Data Transmission

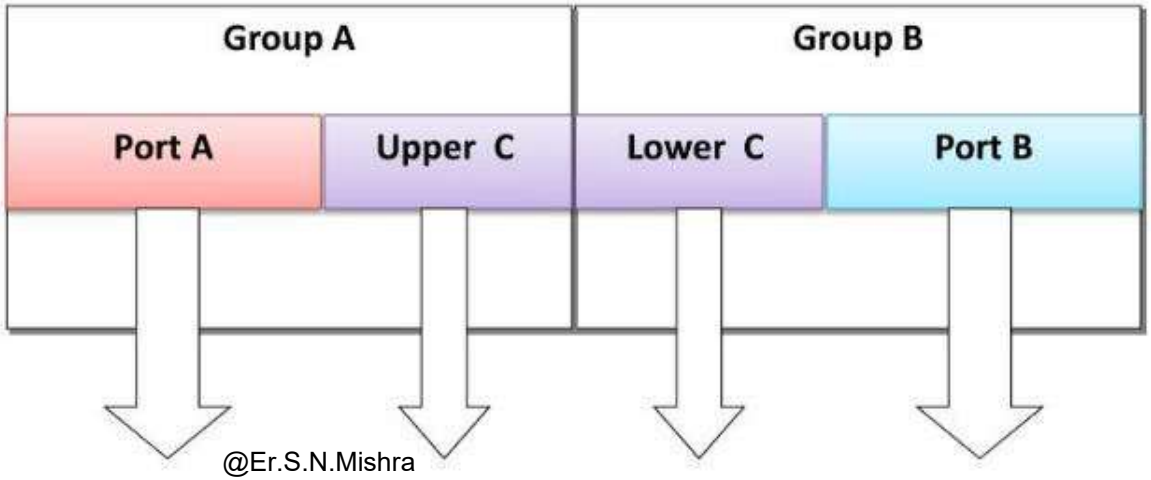
S.N.	Parameter	Asynchronous	Synchronous
1.	Fundamental	Transmission does not based on clock signal	Transmission based on clock signal
2.	Data Format	One character at a time	Group of characters i.e. a block of characters
3.	Speed	Low (< 20 kbps)	High (> 20 kbps)
4.	Framing Information	Start and stop bits are sent with each character.	SYNC characters are sent with each character.
5.	Implementation	Hardware / Software	Hardware

@ITS Wishe

Programmable Peripheral Interface (PPI) - 8255A

- The INTEL 8255 is a device used to parallel data transfer between processor and slow peripheral devices like ADC, DAC, keyboard, 7-segment display, LCD, etc.
- 8255 is a programmable I/O device that acts as interface between peripheral devices and the microprocessor for parallel data transfer.
- In 8255, 24 pins are assigned to the I/O ports. Basically it has three, 8-bit ports that are used for simple or interrupt I/O operations.
- The three ports are **Port A, Port B and Port C** and as each port has 8 lines, but the 8 bits of port C is divided into 2 groups of 4-bit each.
  - ✓ These are given as port C lower i.e., $PC_3 – PC_0$ and port C upper i.e., $PC_7 – PC_4$. And are arranged in group of 12 pins each thus designated as Group A and Group B.

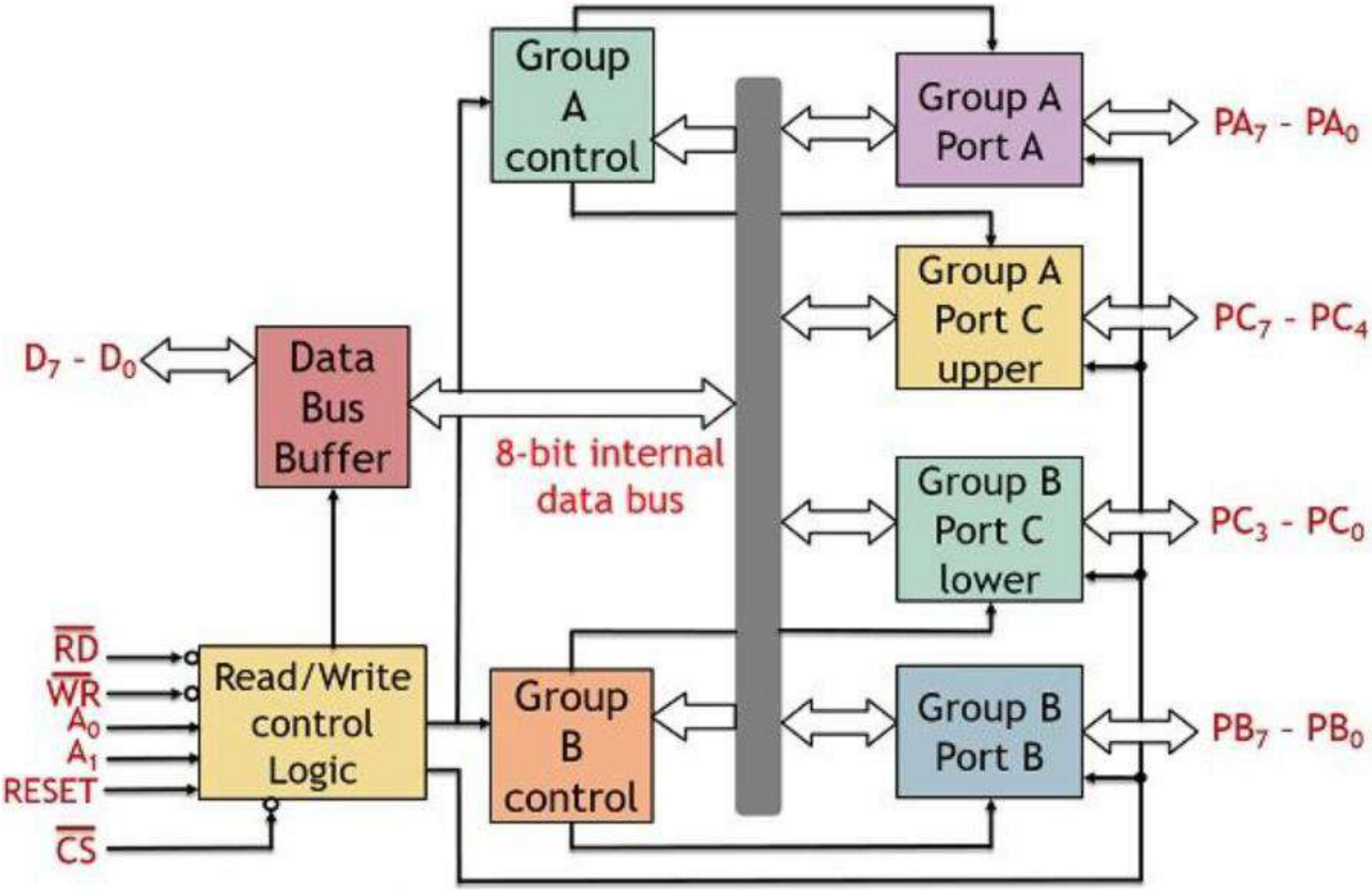
The functions of these ports are defined by writing a control word in the control registers.



Programmable Peripheral Interface (PPI) - 8255A

- Port-A can be programmed to work in any one of the three operating modes mode-0, mode-1 and mode-2 as input or output port.
- Port-B can be programmed to work either in mode-0 or mode-1 as input or output port.
- Port-C (8-pins) has different assignments depending on the mode of port-A and port-B.

Programmable Peripheral Interface (PPI) - 8255A



Architecture of 8255 PPI

Programmable Peripheral Interface (PPI) - 8255A

- If port-A and B are programmed in mode-0, then the port-C can perform any one of the following functions.
  - ✓ As 8-bit parallel port in mode-0 for input or output.
- As two numbers of 4-bit parallel ports in mode-0 for input or output.
  - ✓ The individual pins of port-C can be set or reset for various control applications.
- If port-A is programmed in mode- 1/mode-2 and port-B is programmed in mode-1 then some of the pins of port-C are used for handshake signals and the remaining pins can be used as input/ output lines or individually set/reset for control application.

8255A: Features of Mode-0,Mode-1 and Mode-2

- Mode 0: Ports A and B operate as either inputs or outputs and Port C is divided into two 4-bit groups either of which can be operated as inputs or outputs
- Mode 1: Same as Mode 0 but Port C is used for handshaking and control
- Mode 2: Port A is bidirectional (both input and output) and Port C is used for handshaking. Port B is not used.

Data bus buffer

- It is used to connect the internal bus of 8255 with the system bus so as to establish proper interfacing between the two.
- The data bus buffer allows the read/write operation to be performed from/to the CPU.
- The buffer allows the passing of data from ports or control register to CPU in case of write operation and from CPU to ports or status register in case of read operation.

Read/ Write control logic: Read/ Write control logic

- This unit manages the internal operations of the system.
- This unit holds the ability to control the transfer of data and control or status words both internally and externally.
- Whenever there exists a need for data fetch then it accepts the address provided by the processor through the bus and immediately generates command to the 2 control groups for the particular operation.
- The read/write control logic requires six control signals.
- These signals are given below.
- RD (low): This control signal enables the read operation.
  - ✓ When this signal is low, the microprocessor reads data from a selected I/O port of the 8255A.
- WR (low): This control signal enables the write operation.
  - ✓ When this signal goes low, the microprocessor writes into a selected I/O port or the control register.
- RESET: This is an active high signal.
  - ✓ It clears the control register and set all ports in the input mode.
- CS (low), A0 and A1: These are device select signals.

Operation of the control signals:

A1	A0	SELECTION
0	0	PORT A
0	1	PORT B
1	0	PORT C
1	1	CONTROL REGISTER

A_1	A_0	RD'	WR'	CS'	Input/Output Operation
0	0	0	1	0	Port A - Data Bus
0	1	0	1	0	Port B - Data Bus
1	0	0	1	0	Port C - Data Bus
0	0	1	0	0	Data Bus - Port A
0	1	1	0	0	Data Bus - Port B
1	0	1	0	0	Data Bus - Port C
1	1	1	0	0	Data Bus - Control register

Group A and Group B control:

- These two groups are handled by the CPU and functions according to the command generated by the CPU.
- The CPU sends control words to the group A and group B control and they in turn sends the appropriate command to their respective port.
- As we have discussed that group A has the access of the port A and higher order bits of port C.
- While group B controls port B with the lower order bits of port C.

Mode 0 : Input/ Output mode

- When programmed for MODE 0, the PPI offers three simple I/O ports with no handshaking signals.
- This mode is appropriate for I/O devices that do not need special synchronizing signals to exchange data with the processor.
- A common example is a keyboard used for data entry.
- When used as O/Ps, the PORT C lines can be individually set or reset by sending a special control word to control register address.
- Two halves of PORT C are independent so one half can be initialized as input and the other half as output.

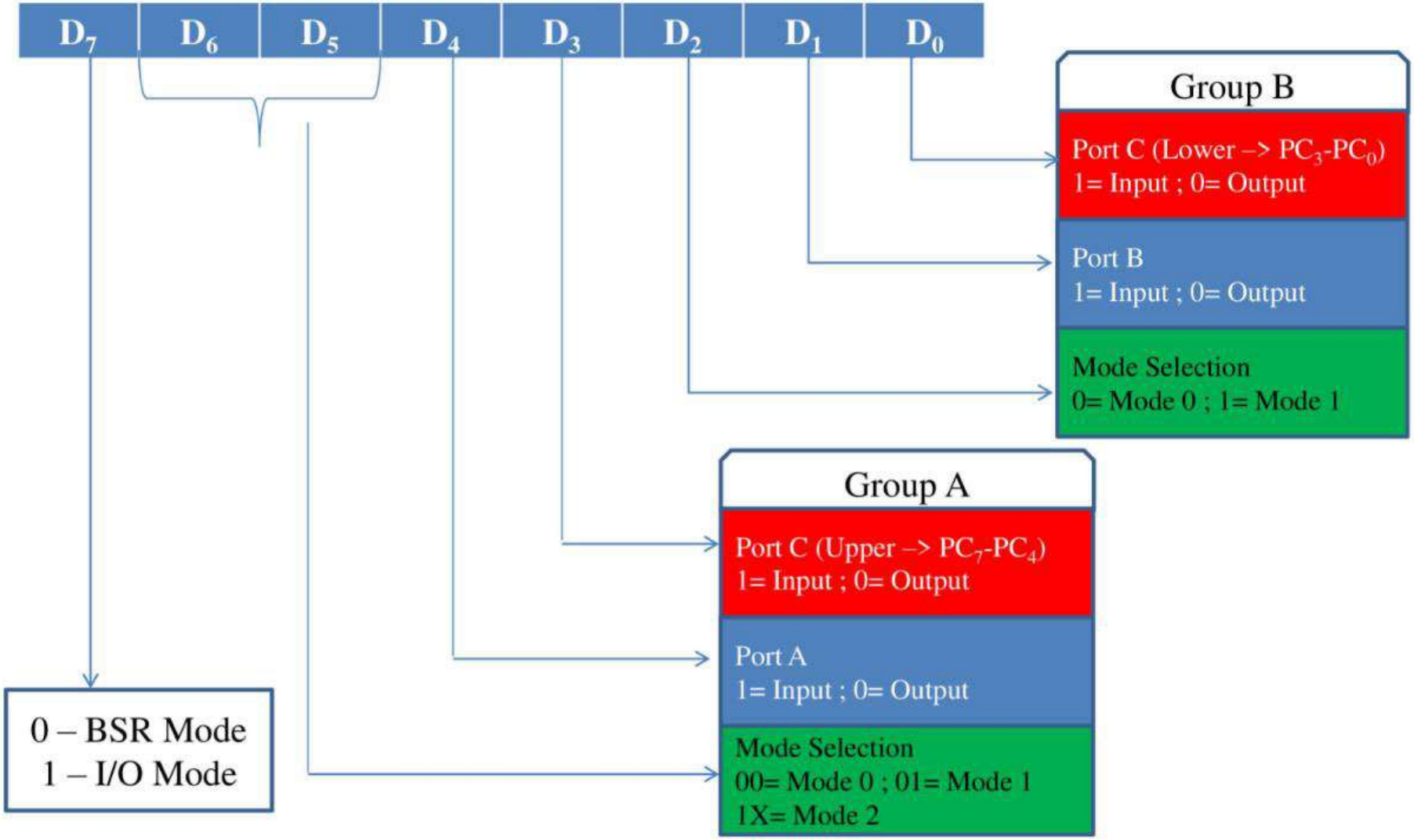
MODE 1: Input/Output with handshaking

- Mode 1 of 8255 supports handshaking with the ports programmed as either input or output mode.
- We know that it is not necessary that all the time the data is transferred between two devices operating at same speed.
  - ✓ So, handshaking signals are used to synchronize the data transfer between two devices that operates at different speeds.
- When programmed for MODE 1, the PPI offers PORT A or PORT B for a handshake input/output operation.
- Both the port utilizes 3-3 lines of port C for handshaking signals. The rest two lines operates as input/output port.
- Pins PC0,PC1 and PC2 function as handshake lines for PORT B
- Pins PC3,PC4 and PC5 function as handshake signal for PORT A (input).
- Pins PC6 and PC7 are available for use as input/output lines for PORT A
- If PORT A is initialized as handshake O/P port, then PORT C pins PC3, PC6 and PC7 function as handshake signals.
- PORT C pins PC4 and PC5 are used as input/output lines for PORT A

MODE 2 : Bidirectional I/O port with handshaking

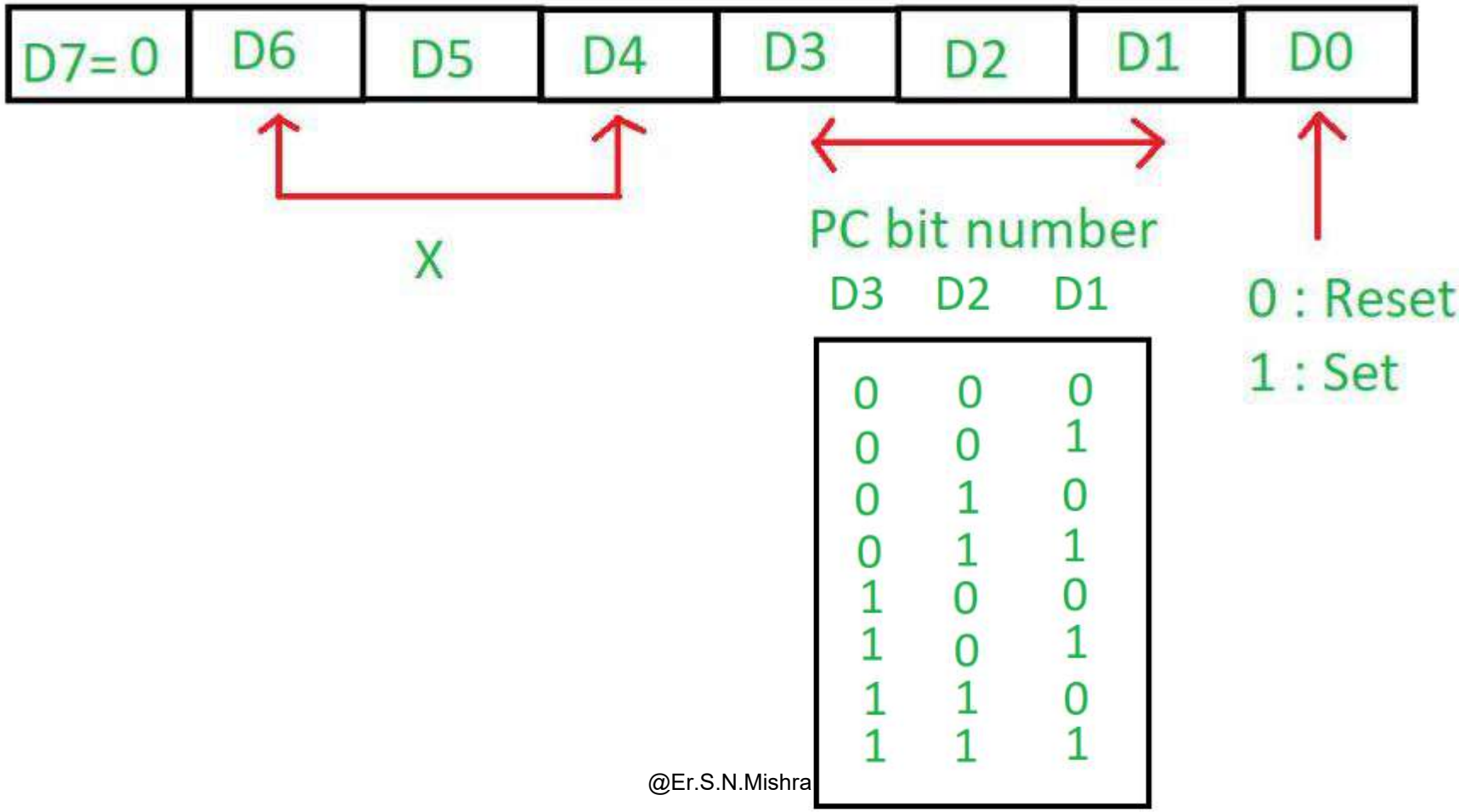
- In this mode, the ports can be utilized for the bidirectional flow of information by handshaking signals.
- Only PORT A can be initialized in MODE 2.
- In MODE 2, PORT A can be used as bidirectional handshake data transfer.
- This means that data can be outputted / inputted from same eight lines.
- If PORT A is initialized in MODE 2, then pins PC3 through PC7 are used as handshake lines for PORT A.
- The other three pins PC0 through PC2 can be used for I/O port if PORT B is in MODE 0.
- The same 3 pins will be used for PORT B handshake signals if PORT B is initialized in MODE 1.

Control Words: Mode Set Control Word



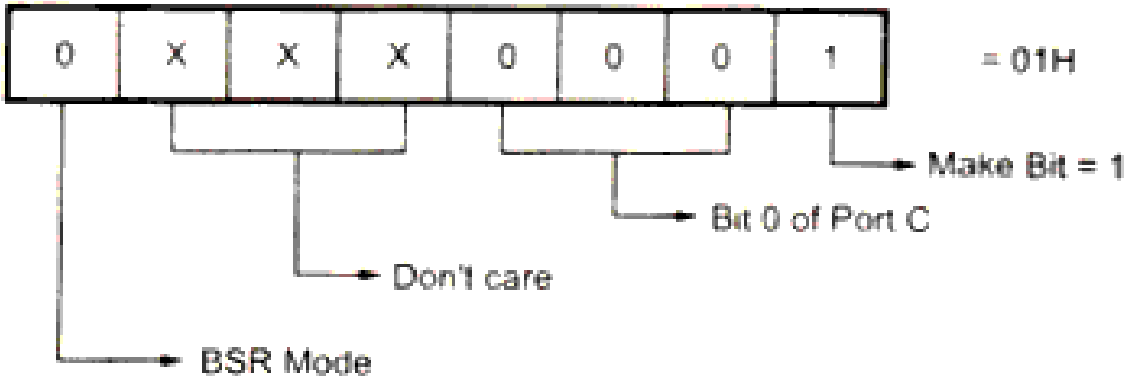
Bit Set Reset (BSR) mode

- This mode is used to set or reset the bits of port C only, and selected when the most significant bit (D7) in the control register is 0.
- Control Register is as follows:

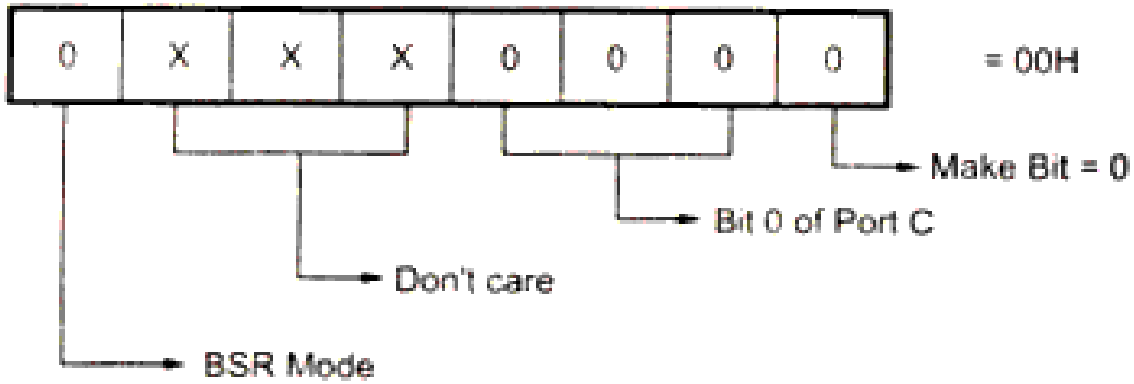


Bit SET/RESET Control Word

Control word to make Bit 0 high



Control word to make Bit 0 low



Control Word: Example

- Obtain the control word when the ports of 8255A are to be used in mode 0 with port-A as output port and port B as input port and port C as output port.
- **Solution.** The control word for this case shown in bellow.

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	
1	0	0	0	0	0	1	0	= 82 H
			A	C_{Upper}		B	C_{Lower}	

Programming 8255

- Write a program to read the DIP switches and display the reading from port B at port A and from port C_L to port C_U

Since Chip select would occur when A15 is high and assuming all other don't care are at logic 0, the port address would be:

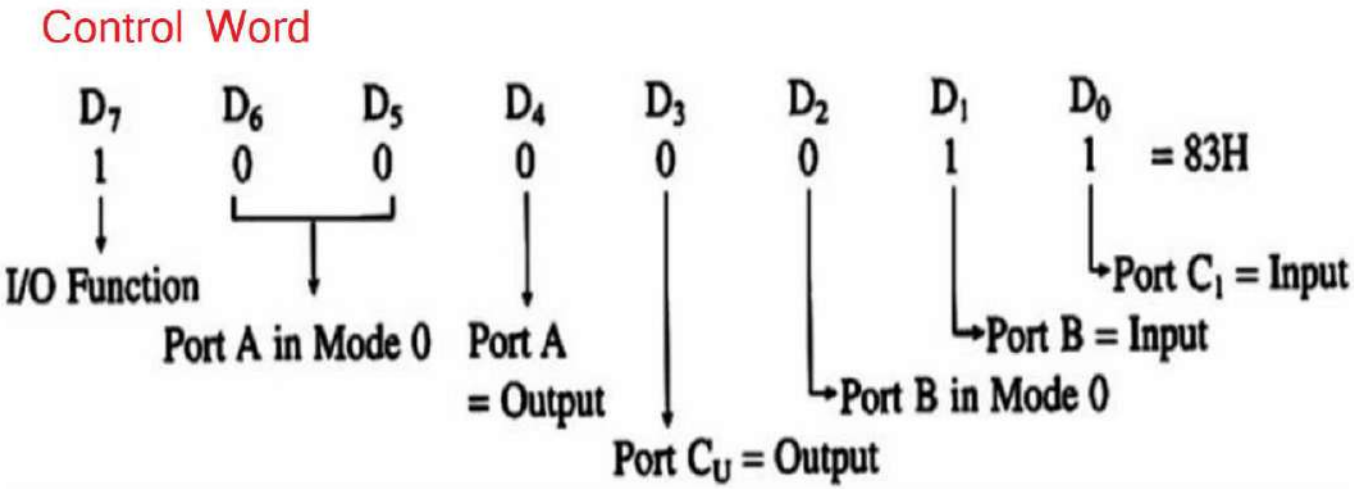
A15	A14.....A2	A1	A0	
1	0000000000000	0	0	: 8000H : Port A
1	0000000000000	0	1	: 8001H : Port B
1	0000000000000	1	0	: 8002H : Port C
1	0000000000000	1	1	: 8003H : Control Register

Port address: Port A: 8000h ($A_1 = 0, A_0 = 0$)
Port B: 8001h ($A_1 = 0, A_0 = 1$)
Port C: 8002h ($A_1 = 1, A_0 = 0$)
Control Register: 8003h ($A_1 = 1, A_0 = 1$)

Programming 8255

Program

```
MVI A, 83H
STA 8003H
LDA 8001H
STA 8000H
LDA 8002H
ANI 0FH
RLC
RLC
RLC
RLC
STA 8002H
HLT
```



Universal Synchronous Asynchronous Receiver Transmitter (USART) – 8251A

- 8251 universal synchronous asynchronous receiver transmitter (USART) acts as a mediator between microprocessor and peripheral to transmit serial data into parallel form and vice versa.
- It takes data serially from peripheral (outside devices) and converts into parallel data.
- After converting the data into parallel form, it transmits it to the CPU.
- Similarly, it receives parallel data from microprocessor and converts it into serial form.
- After converting data into serial form, it transmits it to outside device (peripheral).

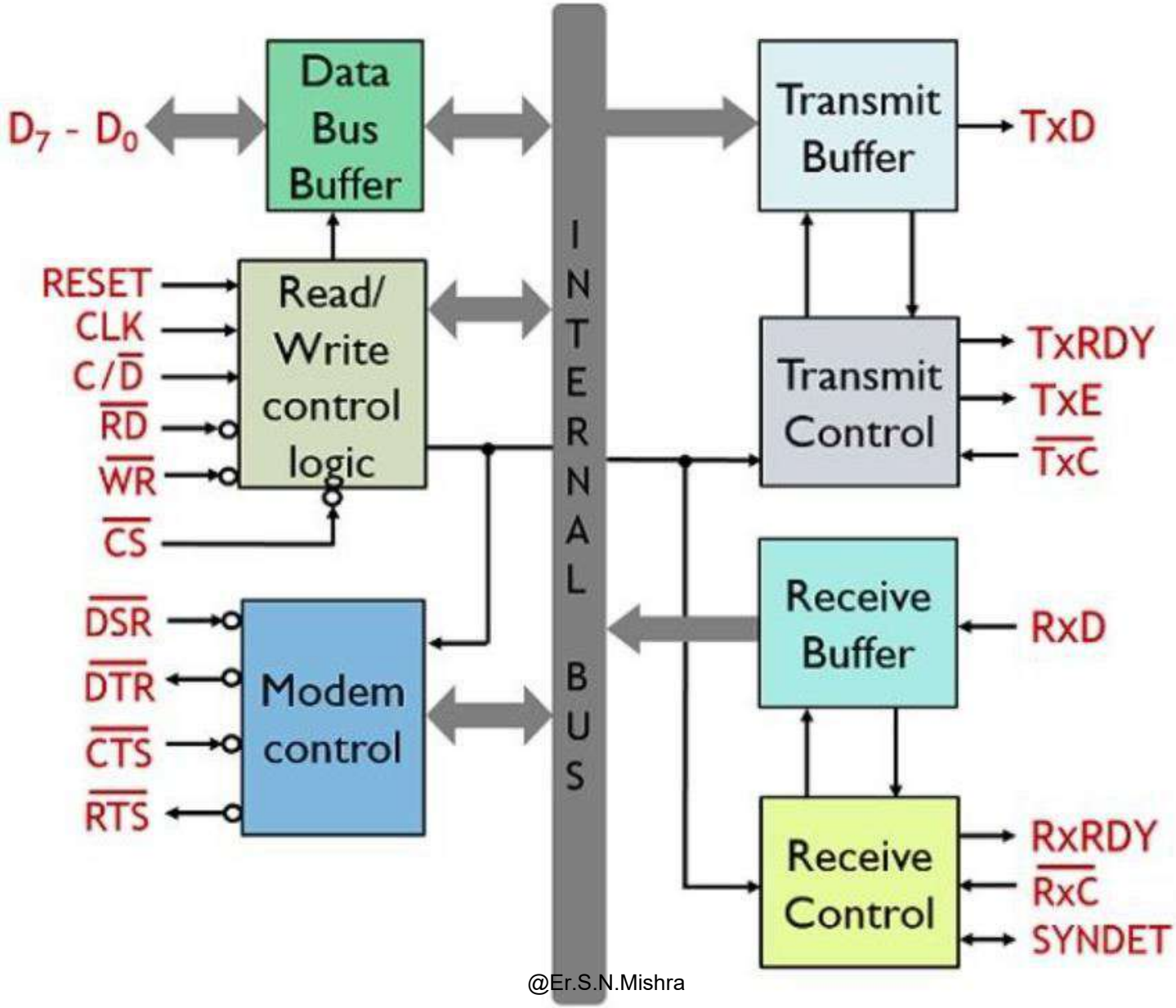
Features of 8251A

- Wide power supply voltage range from 3V to 6V
- Wide temperature range from -40° C to 85° C
- Synchronous communication up to 64 Kbaud
- Asynchronous communication up to 38.4 Kbaud
- Transmitting / Receiving operations under double buffered configuration.
- Error detection capability (Parity, Overrun and Framing)

8251A

- The functional block diagram of 8251A consists five sections. They are:
- Read/Write control logic
- Transmitter
- Receiver
- Data bus buffer
- Modem control

Functional block diagram of 8251A



Data bus buffer

- This block helps in interfacing the internal data bus of 8251 to the system data bus.
- The data transmission is possible between 8251 and CPU by the data bus buffer block

Read/Write control logic:

- The Read/Write Control logic interfaces the 8251A with CPU, determines the functions of the 8251A according to the control word written into its control register.
- It monitors the data flow.
- This section has three registers and they are control register, status register and data buffer.
- The active low signals RD, WR, CS and C/D(Low) are used for read/write operations with these three registers.
- When C/D(low) is high, the control register is selected for writing control word or reading status word.
- When C/D(low) is low, the data buffer is selected for read/write operation.
- When the reset is high, it forces 8251A into the idle mode.
- The clock input is necessary for 8251A for communication with CPU and this clock does not control either the serial transmission or the reception rate.

Read/Write control logic:

$\overline{CS}$	$C/\overline{D}$	$\overline{RD}$	$\overline{WR}$	Operation
1	X	X	X	Invalid
0	0	0	1	data CPU< ----- 8251
0	0	1	0	data CPU ----- > 8251
0	1	0	1	Status word CPU < ----------8251
0	1	1	0	Control word CPU---------- > 8251

Transmitter section: Transmit buffer and Transmit Control

- The transmitter section accepts parallel data from CPU and converts them into serial data.
- The transmitter section is double buffered, i.e., it has a buffer register to hold an 8-bit parallel data and another register called output register to convert the parallel data into serial bits.
- **TXD:** It is an output signal, if its value is one, means transmitter will transmit the data.
- When output register is empty, the data is transferred from buffer to output register. Now the processor can again load another data in buffer register.
- **TXRDY:** It means transmitter is ready to transmit data character.
  - ✓ If buffer register is empty, then TxRDY is goes to high.
- **TXEMPTY:** An output signal which indicates that TXEMPTY pin has transmitted all the data characters and transmitter is empty now.
  - ✓ If output register is empty then TxEMPTY goes to high.
- **TXC:** An active-low input pin which controls the data transmission rate of transmitted data.
  - ✓ The clock signal, TxC (low) controls the rate at which the bits are transmitted by the USART.
- The clock frequency can be 1, 16 or 64 times the baud rate.

Receiver Section: Receive Buffer and Receive Control

- The receiver section accepts serial data and converts them into parallel data.
- The receiver section is double buffered, i.e., it has an input register to receive serial data and convert to parallel, and a buffer register to hold the parallel data.
- **RXD:** An input signal which receives the data.
  - ✓ When the RxD line goes low, the control logic assumes it as a START bit, waits for half a bit time and samples the line again.
  - ✓ If the line is still low, then the input register accepts the following bits, forms a character and loads it into the buffer register.
- The CPU reads the parallel data from the buffer register.
- **RXRDY:** An input signal indicates that it is ready to receive the data.
  - ✓ When the input register loads a parallel data to buffer register, the RxRDY line goes high.
- **RXC:** The clock signal RxC (low) controls the rate at which bits are received by the USART.
- **SYNDET/BRKDET**
  - ✓ During asynchronous mode, the signal SYNDET/BRKDET will indicate the break in the data transmission.
  - ✓ During synchronous mode, the signal SYNDET/BRKDET will indicate the reception of synchronous character

MODEM Control:

- The MODEM control unit allows to interface a MODEM to 8251A and to establish data communication through MODEM over telephone lines.
- This unit takes care of handshake signals for MODEM interface.
- The control circuitry for handing various signals is provided by the modem control unit. It includes DTS, RTS, DTR and CTS.
- These are all active low signals.

✓ ***DSR***: Stands for data set ready and the signal is used to check whether the data set is ready or not when the processor is in the urge of communication.

✓ ***DTR:*** Implies data terminal ready. An active-low signal at this pin shows that the 8251 is now ready to accept the data from the processor.

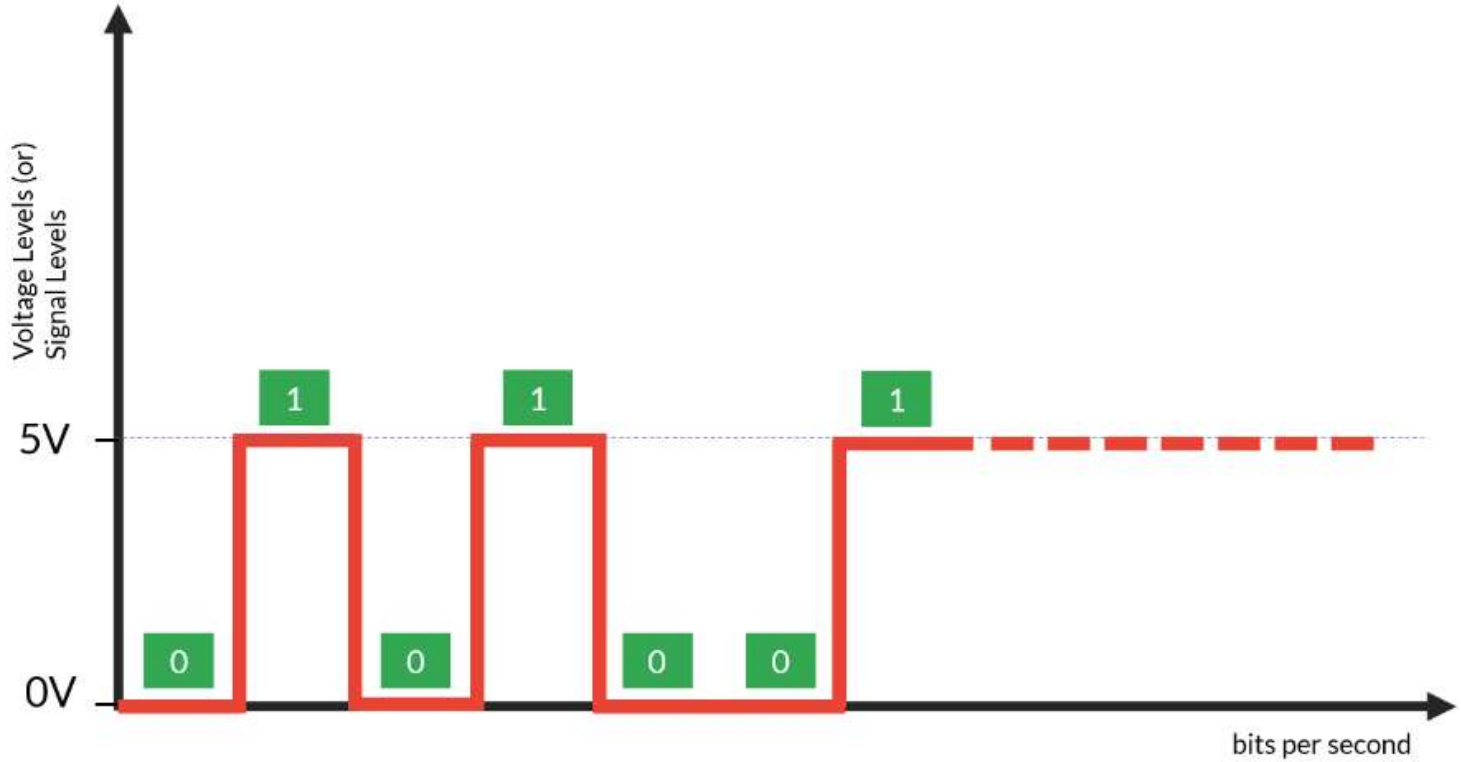
✓ ***RTS***: It stands for the request to send. A low signal shows an assertion for data transmission.

✓ ***CTS***: Clear to send. When 8251 receives a low signal at this pin then it clears all the data present in the modem in order to allow further communication.

Baud rate /Bit rate

- The difference between Bit and Baud rate is complicated and intertwining.
- Both are dependent and inter-related.
- **Bit Rate** is how many data bits are transmitted per second.
- **A baud Rate** is the number of times per second a signal in a communications channel changes.
- Bit rates measure the number of data bits (that is 0's and 1's) transmitted in one second in a communication channel.
- A figure of 2400 bits per second means 2400 zeros or ones can be transmitted in one second, hence the abbreviation bps. Individual characters (for example letters or numbers) that are also referred to as bytes are composed of several bits.
- A baud rate is the number of times a signal in a communications channel changes state or varies.
  - ✓ For example, a 2400 baud rate means that the channel can change states up to 2400 times per second.
  - ✓ The term change state means that it can change from 0 to 1 or from 1 to 0 up to X (in this case, 2400) times per second.
  - ✓ It also refers to the actual state of the connection, such as voltage, frequency, or phase level).

Baud rate/Bit rate

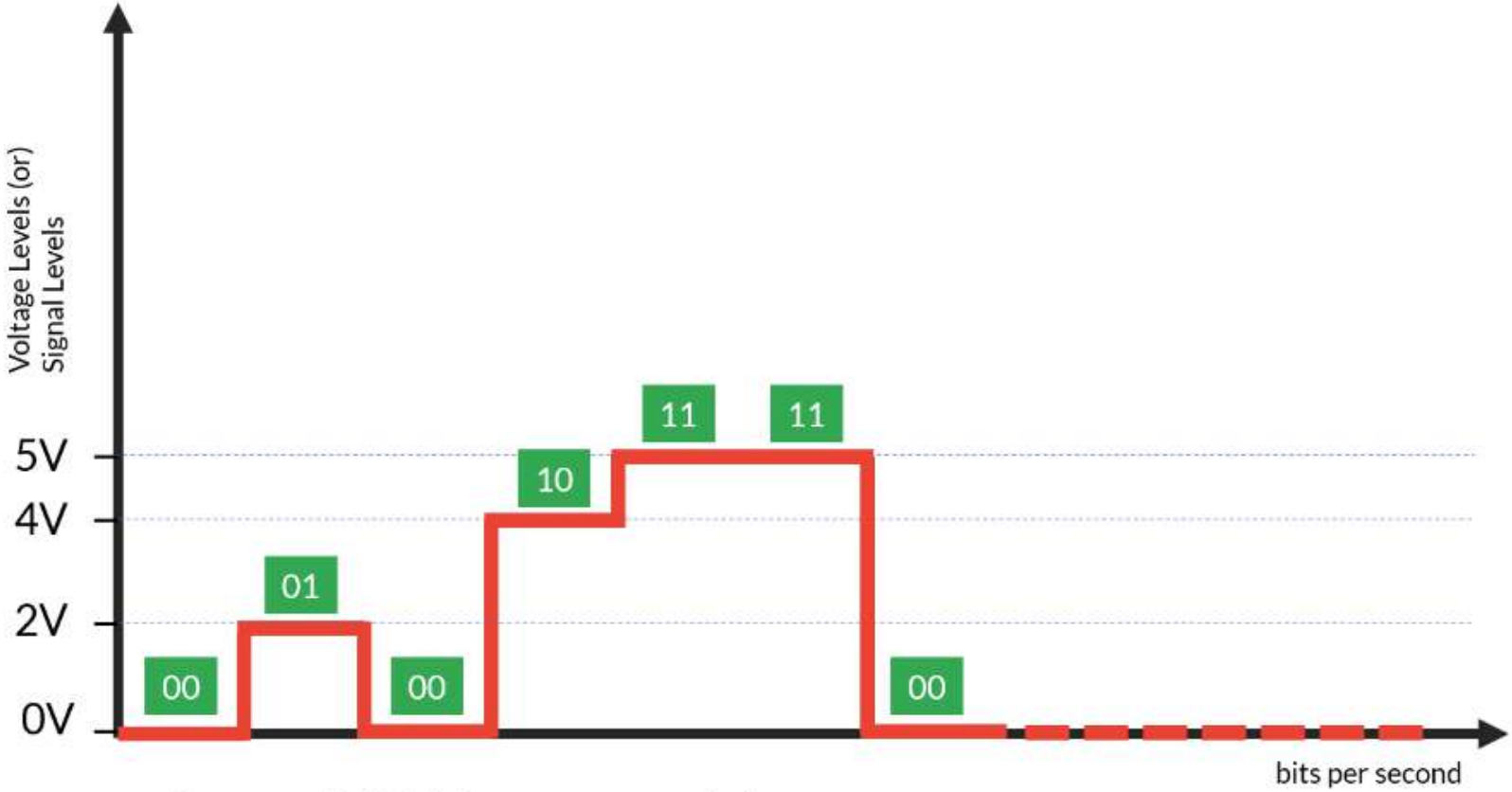


Assume 9600 bits are transmitting
bit rate = 9600 bits/sec
baud rate = 9600 bauds/sec

Baud Rate /Bit rate

- The main difference between the two is that one change of state can transmit one bit, or slightly more or less than one bit, that depends on the modulation technique used.
- So the bit rate (bps) and baud rate (baud per second) have this connection:
- If signal is changing every 10/3 ns then,
- Baud rate = 1/10/3ns = $3/10*10^9 = 3*10^8$ = 300 MBd
- **Note:**
  - ✓ If 1 frame of data is coded with 1 bit then baud rate and bit rate are same.
  - ✓ Sometimes frame of data are coded with two of three bits then baud rate and bit rate are not same.

Baud Rate /Bit rate

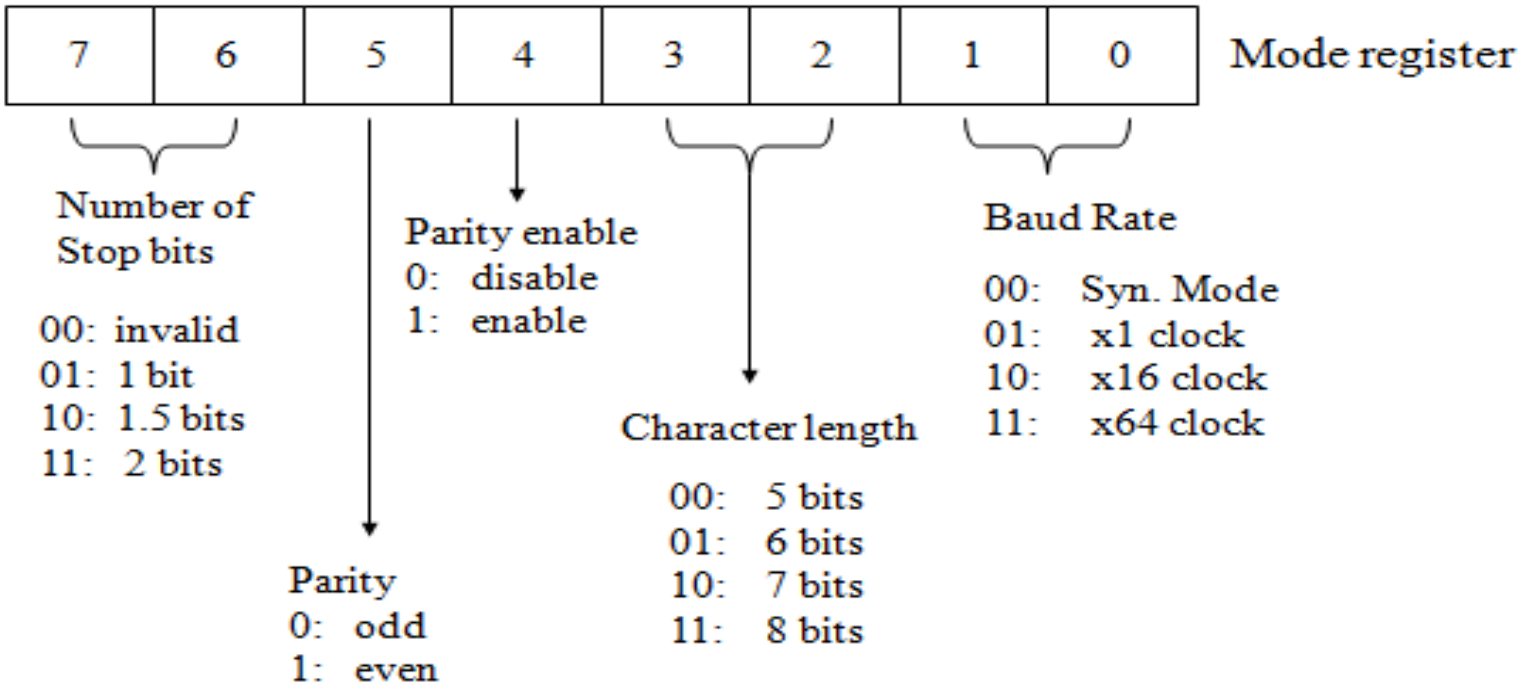


Assume 9600 bits are transmitting
bit rate = 9600 bits/sec
baud rate = 4800 bauds/sec

Baud rate /Bit rate

Bit Rate	Baud Rate
Bit Rate is the number of bits sent per second.	The number of signal units per second is known as the Baud Rate.
The formula of Bit Rate is: = baud rate x the number of bit per baud	The formula of Baud Rate is: = bit rate / the number of bit per baud
Bit Rate is not utilized to determine the amount of bandwidth required for signal transmission.	Baud Rate is used to determine the amount of bandwidth required for signal transmission.
Bit Rate is sometimes defined as the number of bits that move per second.	The number of changes in signal per second is also known as the Baud Rate.
Computer efficiency was prioritized above bit rate.	Baud Rate, on the other hand, is focused on data transfer.

8251 Mode Register

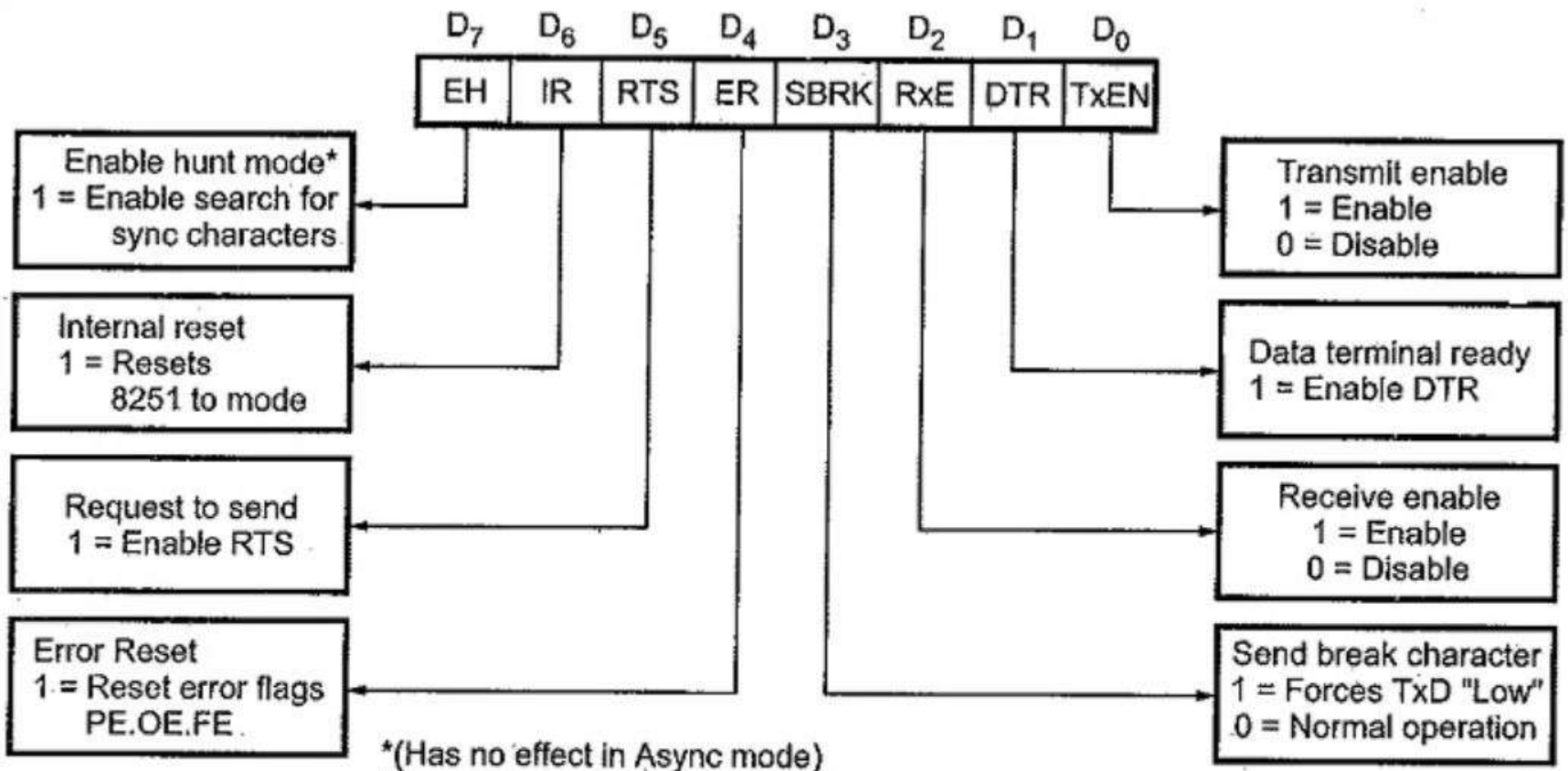


8251 Command Register

EH	IR	RTS	ER	SBRK	RxE	DTR	TxE

command register

TxE: transmit enable
DTR: data terminal ready, $\overline{DTR}$ pin will be low
RxE: receiver enable
SBPRK: send break character, TxD pin will be low
ER: error reset
RTS: request to send, $\overline{CTS}$ pin will be low
IR: internal reset
EH: enter hunt mode



*(Has no effect in Async mode)

Note : Error reset must be performed whenever RX enable and enter hunt are programmed

8251 Status Register

DSR	SYNDET	FE	OE	PE	TxEMPTY	RxRDY	TxRDY

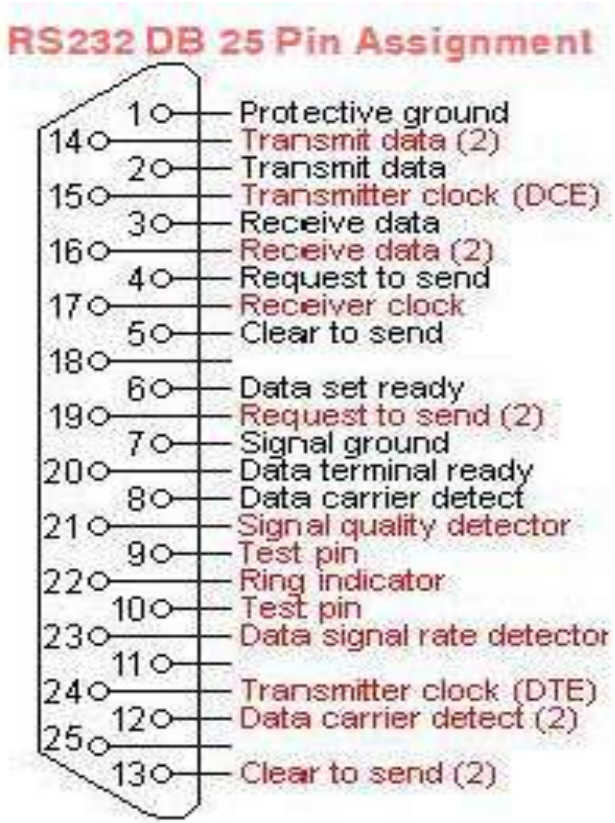
status register

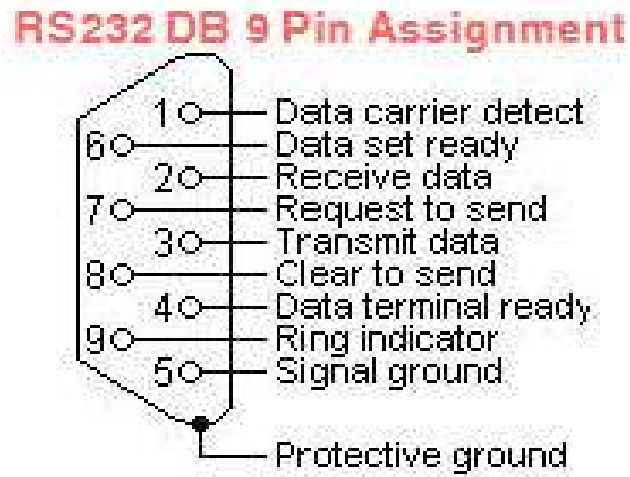
TxRDY:	transmit ready
RxRDY:	receiver ready
TxEMPTY:	transmitter empty
PE:	parity error
OE:	overrun error
FE:	framing error
SYNDET:	sync. character detected
DSR:	data set ready

RS 232C

- This interface standard is most widely used standard for serial communication between microcomputers and peripheral devices.
- The interface, defined by EIA, relates essentially to two types of equipment. The first is known as data terminal equipment, while the second is referred as data communication equipment (DCE).
- The data terminal equipment (e.g a microcomputer) is capable of sending and/or receiving data via the serial interface.
- The data communication equipment on the other hand is generally through of as a device which can facilitate serial data communications (e.g modems).
  - ✓ RS 232C works in a negative logic. The standard specifies that 'the logic one' level is a voltage between -3 and -15 v and 'the logic zero' is a voltage between +3 and +15 V. The commonly used voltages are +12v and -12V.
  - ✓ The transmission line normally used a twisted pair of shielded wire with a line capacitance of more than 1200PF and no less than 300PF. The standard specifies the line length to 50 feet only.
  - ✓ The standard describes the function of 25 signal and handshake pins for serial data transfer. It also specifies that the DTE connector should be male and the DCE connector should be female. Usually 9 pin and 25 pin connectors are available.

RS 232C





The main problem with RS-232C is that it can only transfer data reliably about 50ft (16.4m) at its maximum rate of 20k baud.

If longer lines are used the transmission rate has to be drastically reduced. For higher rate of transfers and for longer distance we have another standard defined.

RS 232C

SERIAL PORT INTERFACE PINOUT AND SIGNALS

9 pin #	25 pin#	Acronym	Full name	Direction	Description
3	2	TxD	Transmit Data	→	Transmits bytes out of computer or device
2	3	RxD	Receive Data	←	Receives bytes into computer or device
7	4	RTS	Request To Send	→	RTS/CTS flow control
8	5	CTS	Clear To Send	←	RTS/CTS flow control
6	6	DSR	Data Set Ready	←	This device is ready to communicate
4	20	DTR	Data Terminal Ready	→	This device is ready to communicate
1	8	DCD	Data Carrier Detect	←	Modem connected to another
9	22	RI	Ring Indicator	←	Ringing on telephone line
5	7	SG	Signal Ground		

Note: DCD can be sometimes labeled as CD

RS 232C interface with DTE and DCE:

- The figure below shows a interfacing with minimum lines

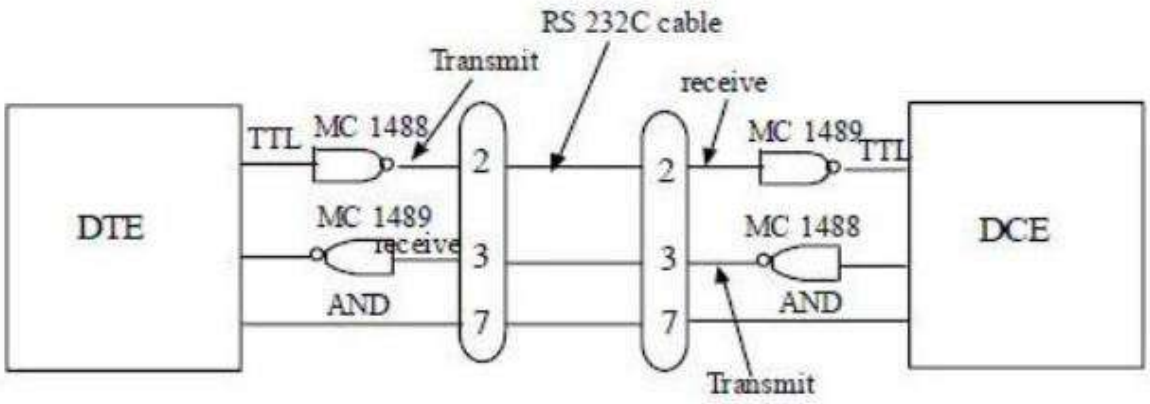


Fig. RS 232C interface

RS 232C Interface with DTE and DCE:

- The signaling in RS 232C is not compatible with the TTL logic level.
- For TTL 0V to 0.2V is considered a logic 0 and 3.4V to 5V as logic 1
- But RS 232C works in a negative logic -3 to -15V considered as logic 1 and +3 to +15V as logic 0.
- Because of this incompatibility of the data lines with the TTL logic, voltage translators called line drivers and line receivers are required to interface TTL logic with RS 232C signals.
- The line driver MC 1488 converts logic 1 into Approx. -9V and logic 0 into +9V.
- Before it is received by the DCE it is again converted by the line receiver MC 1489 into TTL Compatible logic.
- The minimum interface required both a computer and a peripheral device requires three lines: Pin 2,3 and 7.
- These lines are defined in relation to the DTE; the terminal transmits on pin 2 and receives as pin 3.
- On the other hand the DCE transmits on pin 3 and receives on pin 1. Pin 7 is ground pin.

RS-232

Flow	DE-9P	DB-25P	Signal	Description
DTE to DCE	3	2	TxD	Transmitted data
DCE to DTE	2	3	RxD	Received data
DTE to DCE	7	4	$\overline{RTS}$	Request to send
DCE to DTE	8	5	$\overline{CTS}$	Clear to send
DCE to DTE	6	6	$\overline{DSR}$	Data set ready
Common ref	5	7	GND	Signal ground
DCE to DTE	1	8	$\overline{DCD}$	Data carrer detect
DTE to DCE	4	20	$\overline{DTR}$	Data terminal ready

RS-232

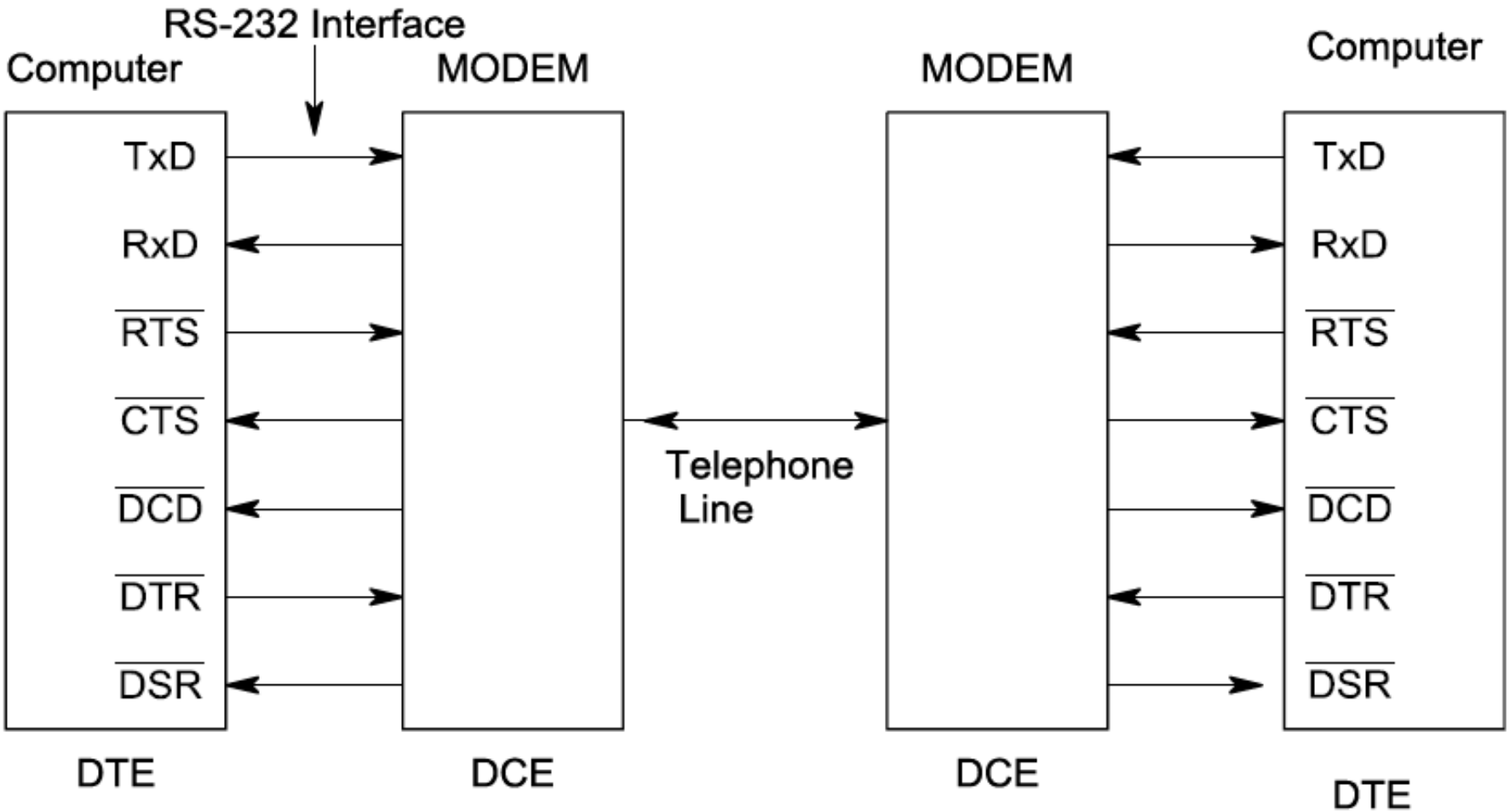
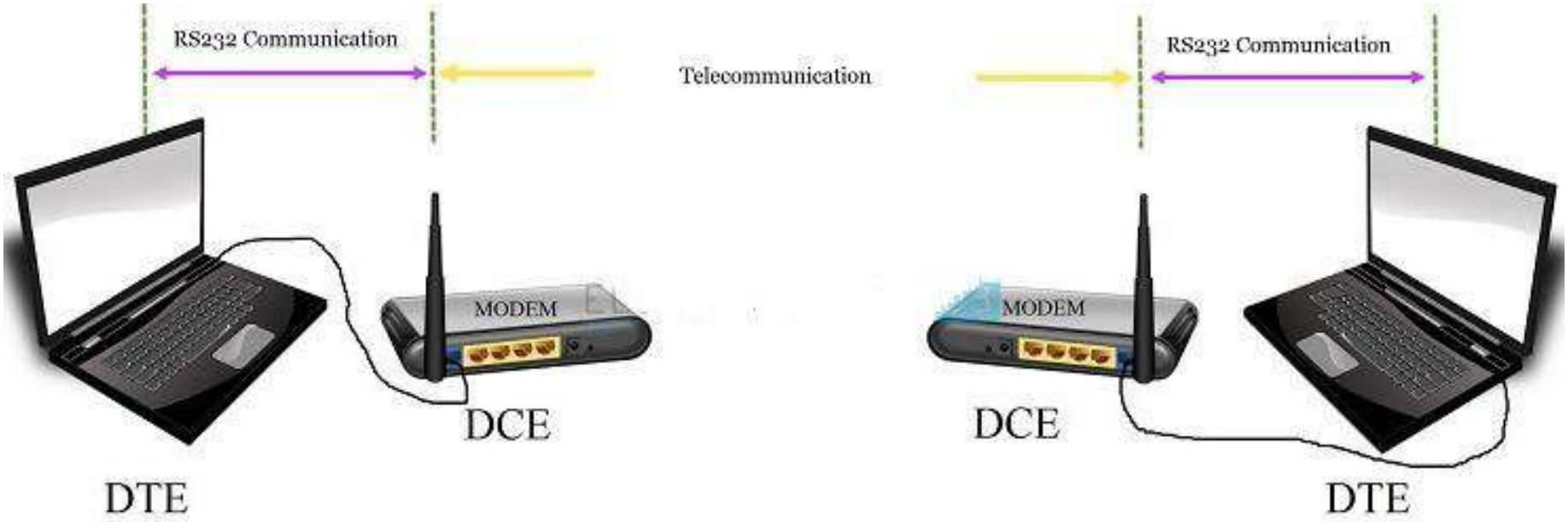


Fig: Digital Data transmission using MODEM and Telephone Line

RS-232


RS232 Communication
Telecommunication
RS232 Communication
MODEM
MODEM
DCE
DCE
DTE
DTE

RS-232

- DTE asserts $\overline{DTR}$ to tell the modem it is ready.
- Then DCE asserts $\overline{DSR}$ signal to the terminal and dials up.
- DTE asserts $\overline{RTS}$ signal to the modem.
- Modem then asserts $\overline{DCD}$ signal to indicate that it has established connection with the computer.
- DCE asserts $\overline{CTS}$ signals, and then DTE sends serial data.
- When sending completed, DTE asserts $\overline{RTS}$ high, this causes modem to unassert its $\overline{CTS}$ signal and stop transmitting similar handshake taken between DCE and DTE other side.
- To communicate from serial port of a computer to serial port of another computer without modem, null-modem is used.

RS-232:Null Modem Connection

- **Null modem** is a communication method to directly connect two DTEs (computer, terminal, printer, etc.) using an RS-232 serial cable.
- The name stems from the historical use of RS-232 cables to connect two tele printer devices or two modems in order to communicate with one another; *null modem* communication refers to using a crossed-over RS-232 cable to connect the teleprinters directly to one another without the modems.
- It is also used to serially connect a computer to a printer, since both are DTE, and is known as a *Printer Cable*.

RS-232:Null Modem Connection

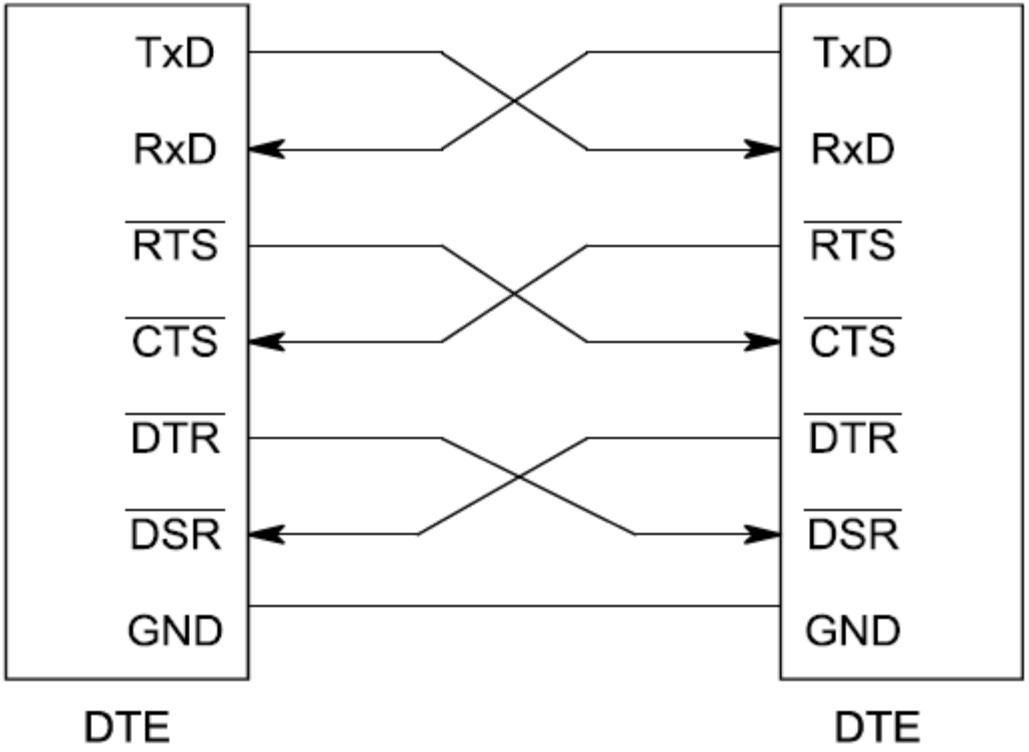


Fig: Null MODEM Connection for RS-232 Terminals